

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JS - Asincrono - Async - Await - ERROR</title>
</head>
<!--
```

El manejo de errores con async/await se realiza principalmente usando bloques try...catch para capturar las excepciones que ocurren durante la ejecución asíncrona, de manera similar a como se hace en el código síncrono. Esto permite detener la ejecución de la operación asíncrona si falla, evitando así que la aplicación falle inesperadamente y facilitando la depuración y gestión de problemas.

```
-->
<body>
  <script>
    /* solución de ERRORES con CALLBACKS
    function sumar(numero1,numero2, callback){
      setTimeout(function(){
        if(typeof numero1 != 'number' || typeof numero2 != 'number'){
          return callback(new Error('Ingreso un valor diferente a tipo numérico'));
        }
        callback(null, numero1+numero2);
      }, 1000);
    }
    sumar('a',20, function(error, resultado){
      if(error){
        console.log(error);
      }else{
        console.log(resultado);
      }
    });
    */

    /* PROMESAS
    function sumar(numero1,numero2){
      return new Promise(function(resolve, reject){
        setTimeout(function(){
          if(typeof numero1 != 'number' || typeof numero2 != 'number'){
            reject(new Error('Ambos argumentos deben ser números'));
          }else{
            resolve(numero1+numero2);
          }
        },2000)
      })
    }

    sumar(30,'b')
    .then(function(resultado){
      console.log(resultado);
    }).catch(function(error){
      console.error(error);
    })
    */

    //ASYNC - AWAIT
    async function sumar(numero1,numero2){
      if(typeof numero1 != 'number' || typeof numero2 != 'number'){
        throw new Error('ALGUN ARGUMENTO NO ES NUMÉRICO');
      }return numero1+numero2;
    }

    async function manejoErrores() {
```